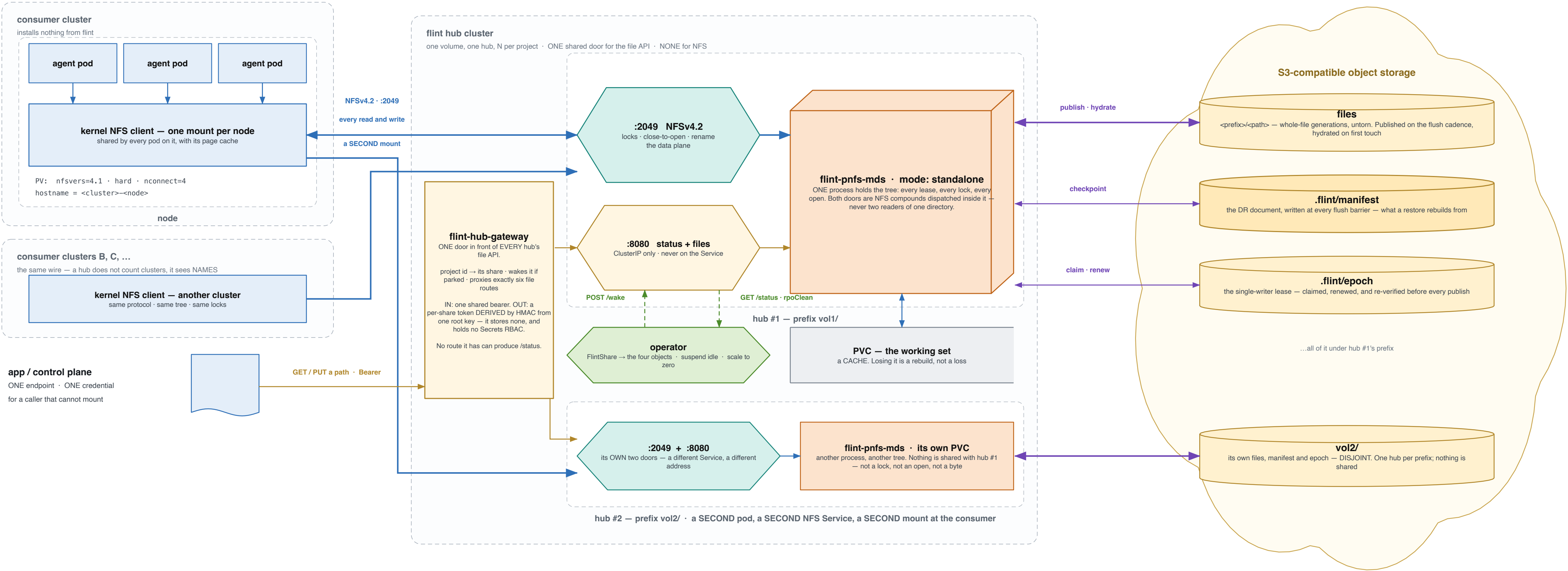


Data flow

One process holds the tree, so N pods in N clusters see one filesystem. The file API can be fronted by one door; the mount cannot.



THE TWO DOORS FRONT DIFFERENTLY, and that is lite's real asymmetry. The FILE API can be multiplexed: flint-hub-gateway resolves a project id to its share, wakes it if it is parked, and proxies six file routes — so a fleet reaches three thousand shares through ONE endpoint and ONE credential, exactly as forge's door fronts every repository. THE MOUNT CANNOT. NFS needs a routable address per hub (spec.service.advertiseAddress → status.address), so a second volume is a second pod, a second Service and a second mount at the consumer. Nothing multiplexes NFS.

AND THE GATEWAY EXISTS BECAUSE PER-SHARE EXPOSURE DOES NOT SCALE. A hub's file API answers at a HEADLESS in-cluster Service the operator deliberately never makes routable: 3000 ClusterIPs exhaust a GKE-default /20 at about 2048 shares — and once the allocator is dry, a new share's CONSUMER Service cannot be created either, so one tenant's share count would stop every other tenant mounting anything — while NetworkPolicy cannot be the guard (off by default, hubNamespaces defaults to [], some CNIs ignore every rule in silence). One gateway can be guarded; three thousand cannot.

ONE KEY, NOT THREE THOUSAND SECRETS. Inbound is one shared bearer, re-read every 10 s so rotation is a Secret edit with no restart. Outbound is per-share and DERIVED — HMAC-SHA256(root, endpoint : bucket : keyPrefix : version) — so the gateway stores no hub credential and has no Secrets RBAC at all; the alternative was `get secrets` in every tenant namespace, where those tenants' S3 keys live. The caller's own credential is never forwarded upstream, and no route the gateway has can produce /status.

ONE PROCESS IS THE COHERENCE AUTHORITY, and that is why each hub is one pod. Every lease, lock and open lives in it, so N pods across N clusters see one tree — held to one by Recreate, an exclusive flock and the RWO attach fence inside the cluster, and by the epoch cell in the bucket across clusters. A second hub on ONE prefix is split-brain: it loses the CAS and is fenced. Two hubs on two prefixes, as drawn, is the ordinary configuration.

A HUB DOES NOT COUNT CLUSTERS — IT SEES CLIENT NAMES. The NFSv4 client identity is the hostname and nothing else, so two clusters sharing a node name are ONE client, and RFC 8881 §18.35.5 requires the server to read the second as the first rebooting: the incumbent's locks and opens go, silently, correctly, on a false premise. Set hostname = <cluster>-<node>, or nfs.nfs4_unique_id.

THE DISK IS A CACHE AND THE BUCKET IS THE COPY, so the recovery point is the flush cadence. Cold files evict at a disk watermark and hydrate on first touch, and both doors SAY SO rather than hanging: NFS parks the caller with NFS4ERR_DELAY, HTTP answers 503 + Retry-After. A file request on a parked share WAKES it — measured at 11 s on kind — which is why the gateway watches the CR and never polls a hub: a poll counts as activity and would pin every share it touched awake.

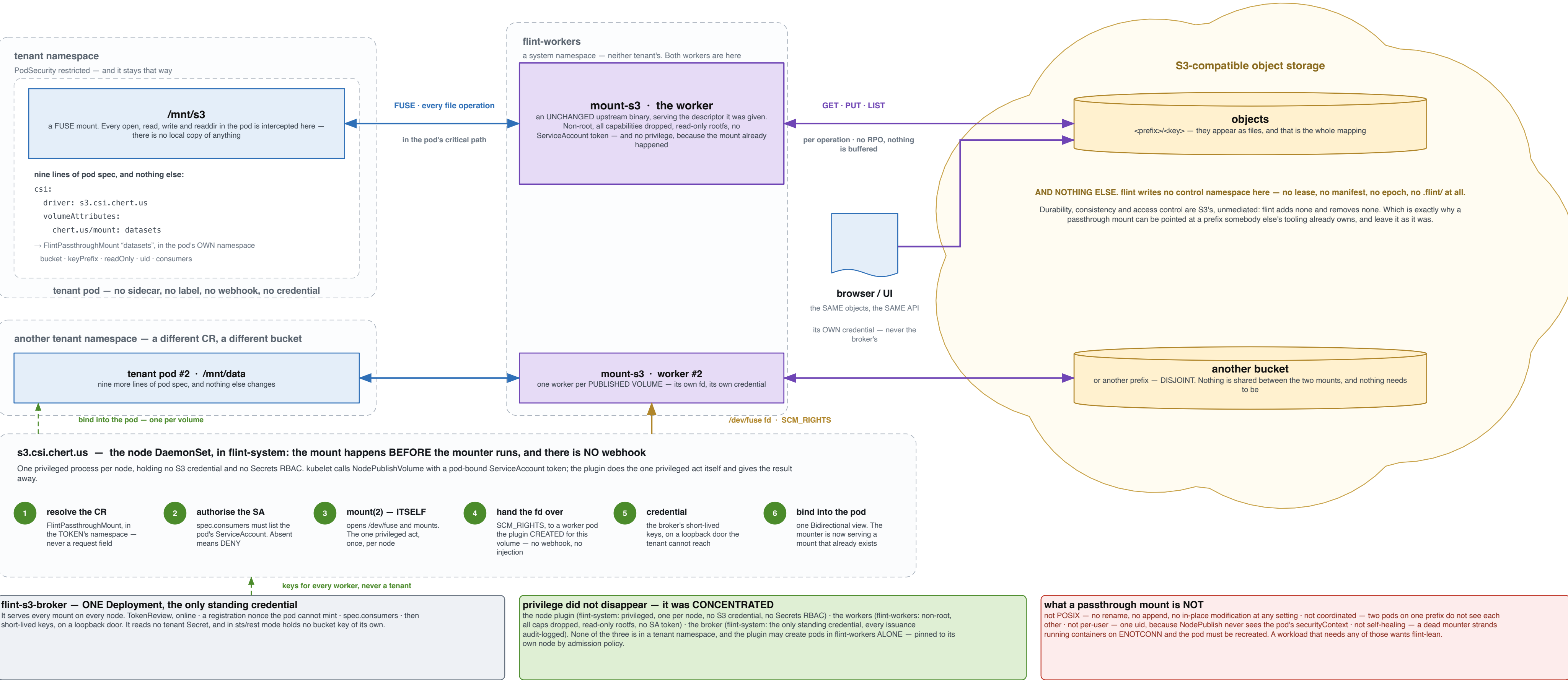
REACHABILITY IS THE BOUNDARY ON :2049 — it authenticates nobody, and kube-proxy SNATs a remote client to an address in the hub's own cluster before the packet arrives (1486 of 1486 connections on a three-cluster rig), so networkPolicy cannot draw it: use peering, security groups or a gateway. And nothing here is a complete oracle of ACTIVITY: the gateway sees every file-API call, but consumers dial each hub's :2049 directly, so a partitioned cluster's busy agents look exactly like idle ones and POST /wake has to be driven by whatever actually knows work is happening.

What the abbreviations mean

NFSv4.2 one protocol, one long-lived TCP flow, and a client already in every node's kernel	close-to-open NFS's coherence rule: what a writer closed, the next opener sees	AUTH_SYS the flavour port 2049 speaks: the client asserts its own uid, and nothing checks it	nconnect how many TCP connections one mount opens. Below 2 there is no trunking to spread
NFS4ERR_DELAY the server telling a caller to come back — how a cold read parks instead of hanging	epoch cell the object that says who the hub is. A second hub loses the CAS and is fenced	flush cadence how often closed generations publish. It is the recovery point, stated as a number	hydrate fetching an evicted file back from the bucket on first touch, into its own inode
evict · watermark cold files leave the PVC at a disk watermark — a marker is written before the truncate	.flint/manifest the DR document: the metadata checkpoint a restore rebuilds the tree from	rpoClean /status answering "can the bucket rebuild this volume right now?" — ask it before deleting a PVC	FlintShare the custom resource one lite volume is declared as. One volume, one hub, N of them per project
flint-hub-gateway the proxy that fronts every hub's file API: one endpoint, one inbound credential	headless Service a Service with no ClusterIP — addressable inside the cluster, routable from nowhere	derived token a per-share credential computed from one root key by HMAC, so nothing has to store or fan out 3000 of them	project id · volume id the labels a share is addressed by through the gateway, instead of by its Service name
Recreate the Deployment strategy that stops the old pod before it starts the new one	RWO attach fence the CSI attachment that makes two hub pods on one PVC impossible, not merely unlikely	nfs.nfs4_unique_id the client-side kernel parameter that gives a node a client identity a rename survives	capture · generation a mutation-complete record of what changed, and the closed unit a flush publishes

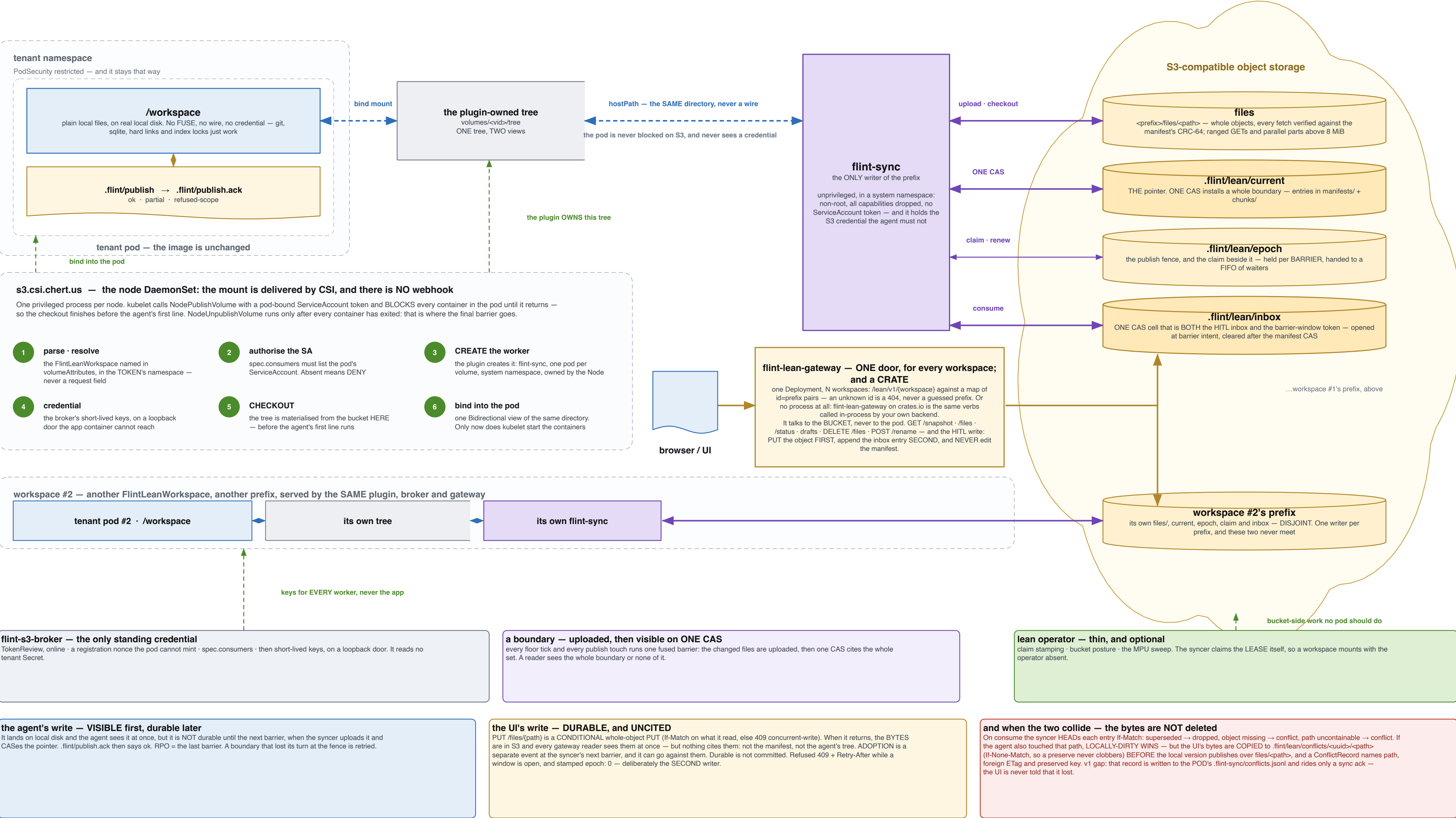
Data flow

Every file operation in the pod is an S3 request. Nothing is buffered, flint writes nothing of its own into the bucket — and the privilege that makes the mount is not in the tenant's namespace.



Data flow

Nothing sits between the pod and its files. A worker beside it — holding the credential the pod must not — reconciles the same directory with the bucket at a boundary the agent declares.



THE ORDERING IS THE ARCHITECTURE. kubelet blocks every container in the pod on NodePublishVolume, so the checkout completes before the agent's first line runs — no init container, no readiness dance — and calls NodeUnpublishVolume only after every container has exited, which is where the final barrier goes. In between, the app touches plain files on real local disk with zero interception.

THE MOUNT IS DELIVERED BY CSI, AND THERE IS NO WEBHOOK. The tenant pod declares a csi: volume naming a FlintLeanWorkspace in its own namespace and stays admissible under PodSecurity restricted: no injected sidecar, no mutating admission, no cert bootstrap, no privileged container and no S3 credential anywhere in the tenant namespace. Privilege is concentrated instead — the node plugin (one per node, holding no S3 credential and no Secrets RBAC), the worker (non-root, no ServiceAccount token) and the broker (the only standing credential, every issuance TokenReview-verified and audit-logged).

THE BOUNDARY VERBS ARE FILES, because the workspace is the interface. An agent that can write a file can declare a coherent point — echo > .flint/publish — and learn when its bytes are durable, from .flint/publish.ack: ok means the bytes are in S3, partial names what a foreign write kept out, and a boundary that lost its turn at the fence is retried rather than refused. No client library, no credential, no network path.

A UI IS POWERED BY THE BUCKET, NOT BY THE POD, and that is forced rather than chosen: the workspace tree is local disk that dies with the pod, and the pod may not exist at all, so there is nothing for a UI to call. flint-lean-gateway — opt-in, its own bearer, an explicit workspace map, and deliberately NOT the lite gateway — reads and writes the same CAS cells the syncer uses, and since 0.1.0 it is a CRATE too: a backend depends on flint-lean-gateway and calls the verbs in-process, holding nothing but credentials for the prefixes it serves. GET /snapshot returns {manifest, manifest_etag, inbox} in one read; GET /files(path) reads guarded on the citation and consults the inbox only when that fails, so an overwrite reads at once and the common read costs nothing extra; delete and rename are DECLARED removals the syncer performs at its barrier (a rename is one generation; a removal of a dirty path is refused, never retried); drafts are durable edits nobody sees until promoted.

THE HITL WRITE IS THREE ORDERED STEPS AND THE MANIFEST IS NOT ONE OF THEM. PUT /files(path) writes the OBJECT first and appends an INBOX entry second — never a manifest edit — so the syncer stays the only writer of the pointer. The worker consumes the inbox at its next barrier (HEAD each entry If-Match; a superseded entry is dropped, not an error), materialises it into the tree and cites it in the next manifest. A write is refused 409 + Retry-After while a barrier window is open, and every gateway replica reads that window from the CELL rather than from its own memory — which is what makes two stateless replicas safe.

THE WINDOW CANNOT WEDGE, AND A DEPOSED WORKER CANNOT WRITE. The window carries a deadline and a successor epoch may override a stale one, so a dead worker does not block HITL forever; and every worker-facing verb — window/open, window/clear, inbox/drop, manifest — is epoch-validated PER REQUEST, so a write whose claimed epoch is not the cell's current epoch is rejected. Rotation alone leaves that door open, which is exactly what the model's LeanNoEpochCheck mutation proves.

AND ONE VERB IS DELIBERATELY CARRIED, NEVER PERFORMED. "Please publish" from outside the pod is honoured; "please pull" is recorded and left to the agent. A boundary publishes what is already on disk and touches no local file, while a sync re-derives the tree and DELETES local files for remotely-deleted paths — so performing it on a remote's say-so would upgrade what a leaked gateway bearer can do from "publish, plus hand over these N named objects" to "rewrite and delete across a running agent's tree, at my timing, under a scope I choose". v1's recorded limits: whole-object HITL writes under a cap, and one shared bearer.

NOTHING HERE IS INJECTED, AND THERE IS NO WEBHOOK IN ANY OF IT. The flint-sync worker is CREATED by the node plugin during NodePublishVolume — one pod per published volume, in a system namespace, pinned with nodeName so it skips the scheduler, and owned by the Node object so a vanished node garbage-collects it. It is not a container in the tenant's pod and no mutating admission rewrites the tenant's spec: the component is the SYNCER, and flint-sync is its binary. The gateway, the broker and the operator are not injected either — each is an ordinary Deployment installed by Helm, and the gateway is opt-in: its chart REFUSES to render without a token Secret and a workspace map, because an unauthenticated gateway is an open writer to every workspace configured.

WHY THE GATEWAY APPENDS TO AN INBOX AND NEVER CASes THE MANIFEST. Not because a second CAS writer would corrupt it — CAS would arbitrate that perfectly well. Because a manifest write is not a metadata edit, it is a CLAIM ABOUT A TREE: a boundary means "everything ordered-before T", and the entries it cites are one half of a pair whose other half is the syncer's on-disk baseline — classify() derives uploads from "changed vs the baseline" and deletes from "absent twice AND present in the baseline". The gateway has no tree (its root is literally /nonexistent), no baseline and no scan, so it cannot honestly say what a coherent boundary contains. It CAN honestly say "these bytes exist and somebody asked for them", and that is exactly what an inbox entry is: a proposal about content, not a claim about a tree.

CONCURRENT UI WRITERS ARE HANDLED IN TWO PLACES AND MISSED IN A THIRD. The inbox cell is CAS'd with a five-attempt retry, so two browsers appending at once serialise and the loser retries — past five it fails the request rather than dropping the entry. And the queue holds ONE entry per path: a newer write supersedes the queued one, and at consume an entry whose ETag no longer matches the object is dropped as superseded. So one path yields one adoption of the last write, however many browsers raced. The object PUT is conditional as well — but on a FRESH HEAD the gateway takes immediately before writing, which closes the HEAD-to-PUT window and is NOT end-to-end optimistic concurrency: so an overwrite must now NAME what it read: 428 precondition-required without an If-Match, 412 file-changed on a stale one, and the current etag on the 412's own header. That is forge's taxonomy, adopted so the three doors are one shape. Until this landed the route accepted no caller If-Match at all, and two browsers that each read v1 and then wrote BOTH succeeded — the second silently winning.

AND WHY NOT SIMPLY ROUTE TO THE WORKER, the way lite's gateway routes to a hub? Because half the time there is nothing to route TO. A lean worker is created by the node plugin at NodePublishVolume and DELETED at NodeUnpublish after the drain, so a workspace no tenant pod is mounting has no flint-sync anywhere — it is only a prefix in a bucket, and a UI must still be able to list and write it. A lite hub is a Deployment an operator can scale 0→1 and wake; a lean worker is a side effect of a POD being scheduled, and no gateway can conjure a tenant pod. Nor is there a door to route to: flint-sync's control surface is a UDS in the pod's emptyDir, and the code says why it stays that way — "there is no TCP listener and no authentication, because the trust boundary is the pod; a TCP listener would be a new remote surface, which is a stated non-goal".

AND ROUTING WOULD HAND THE UI THE WRONG GUARANTEE, which is the part that surprises. A write placed into the worker's tree is visible at once and durable only at the next barrier — the AGENT's bargain. The UI's 200 would then precede the bytes reaching S3, and a pod that dies in between loses the write. The inbox inverts exactly that: durable when the call returns, adopted later. The same line is already drawn inside the pod — the UDS sync verb EXECUTES, "because the caller is inside the pod: it is the agent asking for its own tree to be updated, which is the agent's own decision to make", while the gateway's sync request is carried and never performed. Inside the pod is a decision; outside it is a proposal.

AND THREE CONSEQUENCES FOLLOW FROM THAT ONE. A manifest write would need the fence the gateway does not hold — it stamps epoch: 0 — so either the gateway queues for the fence like a syncer and runs a merge, or manifest writes stop being fenced, which is the deposed-straggler hole. It would move the pointer's ETag on every UI write, so baseline.manifest_etag would no longer match and the syncer would take its adopt-a-foreign-manifest path on every one of them — an exceptional path made the common path. And it would read-modify-write an entries set that runs to hundreds of MiB at a million files, racing the barrier's own commit. The inbox is a few hundred bytes, needs agreement with nobody, and doubles as the barrier-window token, so "is a barrier in flight?" and "queue this write" are one read and one CAS.

THE GATEWAY IS A WRITER, BUT NOT THE SAME KIND OF WRITER, and the two guarantees are exact opposites: the agent's write is visible before it is durable, the UI's is durable before it is visible. What keeps that safe is that only ONE of them may write the manifest. The gateway stamps epoch: 0 and never edits the pointer; the syncer holds the lease and cites the inbox at its own barrier. So "written" means different things at the two doors, and a UI that reports success on a PUT is reporting durability, never adoption.

TWO WORKSPACES SHARE THE MACHINERY AND NOTHING ELSE. The node plugin is one per node, the broker is one Deployment, and the gateway is one process whose workspace map is a list of id=prefix pairs — an unknown id is a 404, never a guessed prefix, and in v1 ONE bearer covers every workspace in that map, which is the recorded limit to know before pointing a UI at a multi-tenant one. Everything below that is per volume: its own worker pod (one per PUBLISHED volume, created by the plugin on its own node and owned by the Node so a vanished node GCs it), its own tree, its own credential, and its own prefix with its own epoch, claim, pointer and inbox. One writer per prefix is a mechanism inside a product; across products it is only a convention, so what assigns prefixes is what keeps two workspaces apart.

THREE QUESTIONS DECIDE AGAINST LEAN, cheapest disqualifier first: does the tree fit the disk, and its file count the checkout budget (the manifest is chunked and a publish is O(changed), but every checkout still materialises every file); is one writer per subtree enough; is snapshot freshness acceptable. A no on any of them means the hub. And a worker is never taken away from a tenant still using its tree by ORDERING, not by a PodDisruptionBudget — a PriorityClass for kubelet's graceful shutdown and a preStop that waits for the release, because on a spot fleet the eviction API is not involved at all.

-----> the workspace — one directory, two views, never a wire

—————> durable path — the worker is the only writer of the prefix

-----> control plane — never carries a file

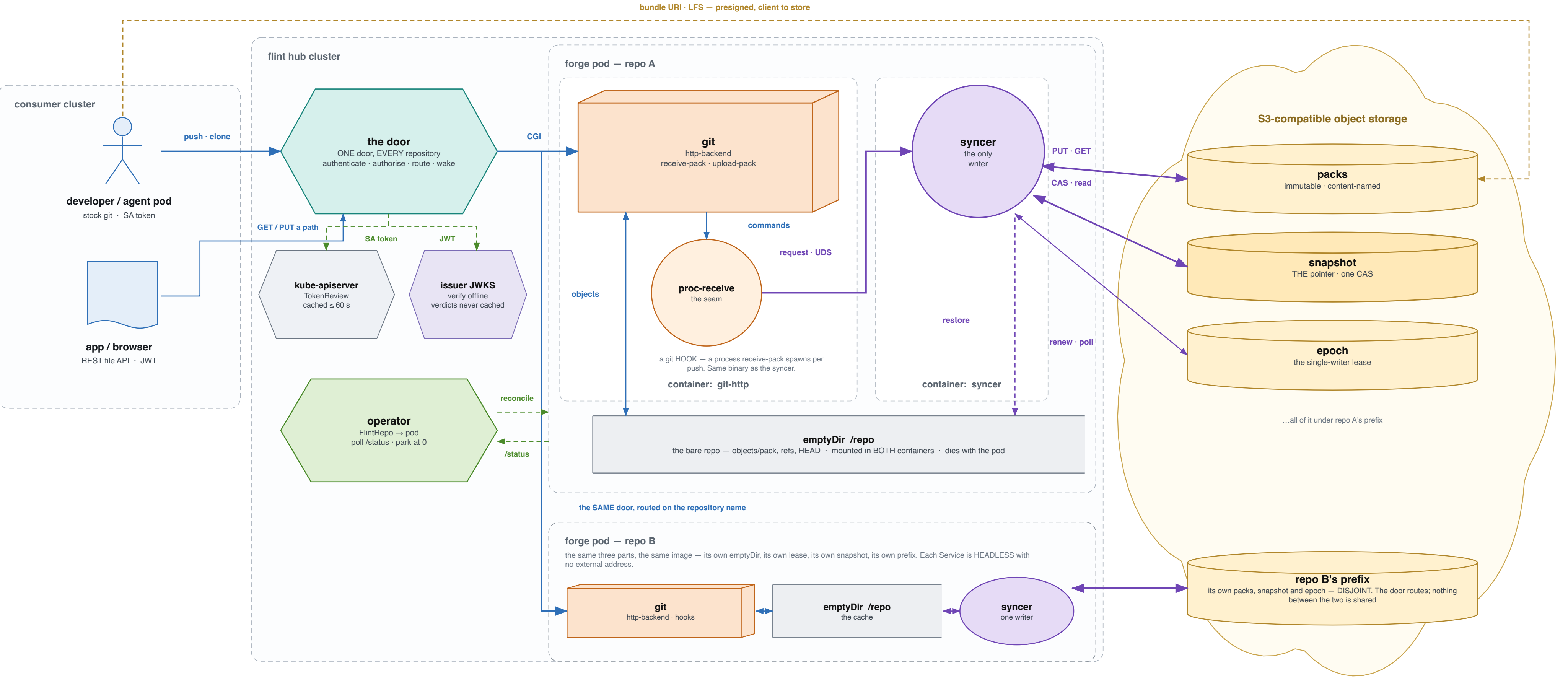
—————> the file protocol — boundary verbs in the tree, REST at the gateway

What the abbreviations mean

CSI ephemeral inline a volume declared in the pod spec itself — no PVC, no PV, nothing to bind beforehand	NodePublishVolume the CSI call kubelet BLOCKS every container on. The checkout happens inside it	NodeUnpublishVolume the call kubelet makes only after every container has exited — the final barrier	bind mount · hostPath two names for ONE directory: the plugin's tree, seen from the pod and from the worker
CAS compare-and-swap — a write that lands only if the object is still the version you read	ETag / If-Match S3's precondition: the ETag you read, offered back as the terms of the write	the pointer .flint/lean/current — the ONE mutable metadata object. Entries live in immutable manifests	fence a lost CAS. Another syncer holds this workspace, so this one stops publishing and says so
epoch · lease the bucket-side single-writer cell. It lives in the BUCKET, so it fences a writer in ANY cluster	claim the project identity stamped on the prefix: adopt your own, refuse a foreign one	boundary verb a file the agent writes to declare a coherent point, and an ack file it reads back	boundary one fused barrier: upload the changed files, then install the whole set with one CAS
loopback door AWS_CONTAINER_CREDENTIALS_FULL_URI: how short-lived keys reach the worker, and only the worker	TokenReview the Kubernetes API that says whether a ServiceAccount token is still valid — online, not offline	FlintLeanWorkspace the custom resource one lean project is declared as, in the tenant's own namespace	RPO recovery point. For lean it is the last BARRIER, never the last write
HITL human in the loop: a write that reaches the workspace from a UI rather than from the agent	the inbox the queue of UI writes waiting to be adopted. Entries name an object and its ETag, never a manifest edit	the window the barrier-in-progress token, in the SAME cell as the inbox — carrying a deadline, so it cannot wedge	epoch-validated checked PER REQUEST against the cell's current epoch, so a deposed worker's write is refused

Data flow

Stock git serves every byte. ONE stateless door fronts every repository; one syncer beside each is the only process that can write S3.



ONE DOOR, MANY REPOSITORIES — and this is exactly where forge and flint-lite differ. The door holds no repository state: it verifies the token, resolves the repository named in the URL to its FlintRepo, routes to that repository's pod and wakes it if it is parked — so a thousand repositories are ONE endpoint, one DNS name and one credential shape to every client. A lite hub cannot be fronted that way, because the hub IS the tree: one process, one prefix, dialled directly at the address its own Service advertises. A second lite volume is a second pod, a second Service and a second mount at the consumer. And the door is not only a router: where the operator renders the NetworkPolicy, it is the ONLY peer admitted to a repository pod's git port — which is what makes X-Remote-User mean anything. Where that policy is NOT rendered, reaching the port IS the authorisation — and the operator raises PrincipalsUnenforced against that repository rather than leaving it unsaid. Either way a client that bypasses the door is already outside the trust model, so the door is a COMPLETE oracle of who pushed and when — which is what the idle ladder counts. A lite hub has no such place.

TWO CREDENTIAL KINDS, SERVED AT ONCE. The door routes on the token's iss: a pod's ServiceAccount token goes to TokenReview at the apiserver, an issuer-minted JWT to offline signature verification against the cached JWKS. A JWT's sub — a person — becomes the principal for spec.consumers, for the branch policy, and as the commit author.

AND THE DOOR REFUSES TO START if the cluster's ServiceAccount issuer and the configured JWT issuer are the same string: every pod token would then route to the offline verifier, which cannot know a pod was deleted.

ONE CAS ON THE SNAPSHOT is the transaction boundary. Told “ok” means that object moved; a 412 on it is a fence, not a retry.

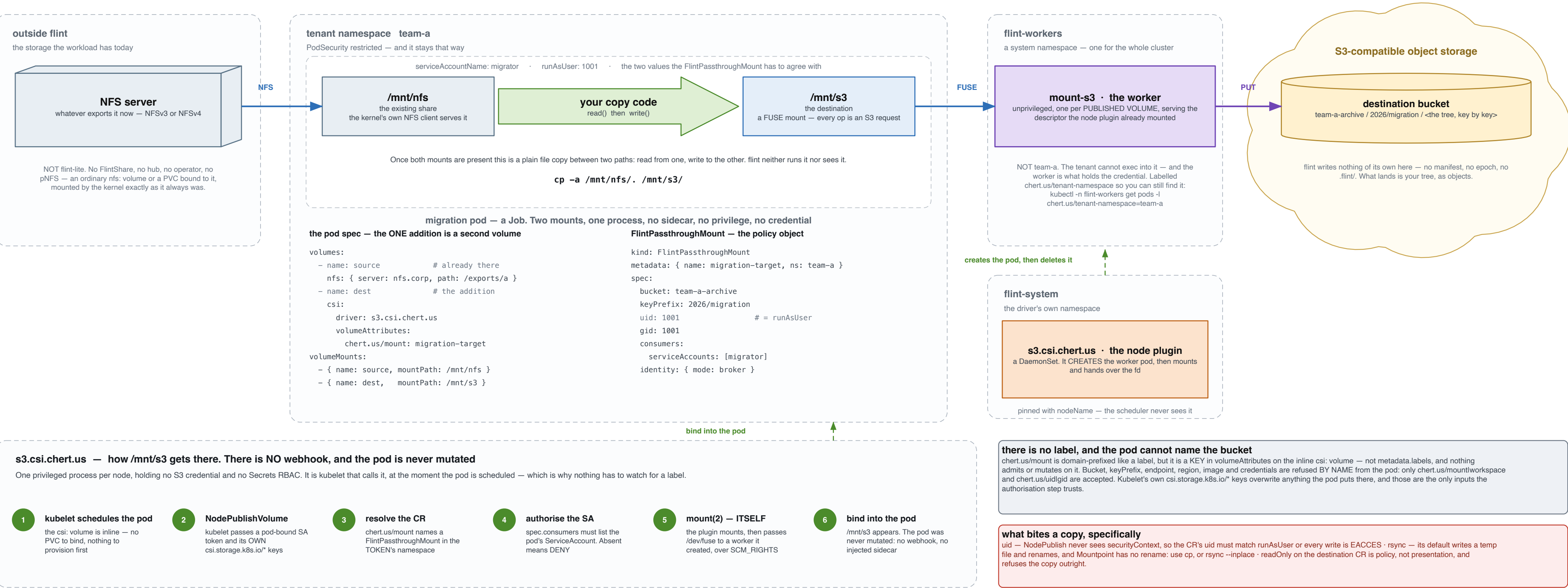
→ data plane — every byte of a clone or push → durable path — the only writer of the bucket - - - - - control plane — never reads an object - - - - - bypass — the pod sees a few hundred bytes

What the abbreviations mean

CAS compare-and-swap — a write that lands only if the object is still the version you read	ETag / If-Match S3's precondition: the ETag you read, offered back as the terms of the write	fence a lost CAS. Another server holds this repository, so this one stops READING too, and exits	CGI Common Gateway Interface — how the runner invokes git http-backend, once per request
UDS Unix domain socket — the hook-to-syncer channel, inside the pod, never over a network	proc-receive the git hook that lets the server, not git, decide each ref update	pre-receive the git hook that applies the branch policy, and can refuse the whole push	emptyDir a Kubernetes volume that lives and dies with the POD — not with a container
SA token a Kubernetes ServiceAccount token, projected into the pod and rotated by the kubelet	TokenReview the Kubernetes API that says whether a ServiceAccount token is still valid	JWT JSON Web Token — a signed bearer token, verified at the door with no round trip	JWKS JSON Web Key Set — the issuer's signing keys. The KEYS are cached, the verdicts are not
iss / sub / exp a JWT's issuer, its subject — the principal — and its expiry	LFS Git Large File Storage — big blobs kept beside the repository, moved by presigned URL	bundle URI git's own opt-in: the server names a pack the client fetches straight from S3	FlintRepo the custom resource that one forge repository is declared as

Migration — an NFS share into a bucket

The pod already mounts its NFS share. A SECOND volume in the same pod spec names a FlintPassthroughMount, and the CSI node plugin mounts the destination bucket at /mnt/s3. The copy itself is ordinary code in the pod — flint moves no bytes of its own.



THE POD IS CREATED THE ORDINARY WAY, and that is the whole point of the shape. You apply a Job whose spec has two volumes: the nfs: one it already had, and a csi: one naming a FlintPassthroughMount. There is no mutating webhook in flint at all, so nothing rewrites the pod between kubectl and kubelet — what you applied is what runs. The mount appears because kubelet calls NodePublishVolume on the node plugin when it schedules the pod, and the plugin does the one privileged act itself. A typo in chert.us/mount is therefore not an admission error but a FailedMount event naming the CR and the namespace, and the pod simply never starts.

IT USED TO BE A LABEL, and anyone who set this up before v1.45.0 will look for one. Until then a mutating webhook watched for flint.io/passthrough-mount on the pod — objectSelector, key Exists — and the label's VALUE named the CR, exactly as the volumeAttributes value does now. What it injected was a PRIVILEGED native sidecar, and that privilege was not reducible: /dev/fuse alone would take CAP_SYS_ADMIN and a hostPath device, but the mount has to REACH the app container, which needs mountPropagation: Bidirectional, which the API server permits only on a privileged container. So a namespace enforcing PodSecurity baseline or restricted REJECTED the mutated pod — correctly. 'fcac038f' replaced both sidecar-injection webhooks (passthrough's and lean's) with the node DaemonSet, because spec.volumes[*].csi is on the restricted allow-list. The information you supply did not change; its carrier did.

THE WORKER IS A POD, AND IT IS NOT IN YOUR NAMESPACE. It lives in flint-workers — one system namespace for the whole cluster, not one per tenant — created by the node plugin during NodePublishVolume, pinned to this node with nodeName so the scheduler never places it, and owned by the Node object so a vanished node garbage-collects it. Three reasons it is not in team-a: the worker is what holds the credential, and anyone who can exec into pods in their own namespace could read it; PodSecurity is enforced differently, since flint-workers must be labelled privileged (the lean tree needs hostPath, forbidden under baseline) while your namespace stays restricted; and the plugin's own RBAC is a Role in flint-workers ALONE — pods create/get/list/watch/delete there and nowhere else, with a ValidatingAdmissionPolicy that further pins spec.nodeName to the node making the call, so a compromised node agent cannot place a pod anywhere but on itself. You can still find yours: chert.us/tenant-namespace and chert.us/tenant-pod are labels on the worker.

THE POD NAMES A CR; THE CR NAMES THE BUCKET. That indirection is the security property worth drawing, because it is what makes the destination a decision the platform owns rather than one the workload asserts. The pod author cannot point a mount at an arbitrary bucket by editing their own manifest — volumeAttributes are attacker-controlled input to a privileged process, and every key but the selector and the two integers is refused by name. Whoever may create a FlintPassthroughMount in that namespace decides which buckets exist to be named, and spec.consumers decides which ServiceAccounts may name them.

THE COPY IS YOUR CODE, running between two paths that happen to be backed by very different things: /mnt/nfs is served by the kernel's own NFS client and /mnt/s3 by a FUSE mount whose every operation is an S3 request. Nothing coordinates them and nothing needs to — the process reads a file and writes a file. What that costs is the S3 round trip in the copy's critical path, so throughput comes from concurrency: several files in flight, or several pods each given a subtree. What it saves is everything else — no staging disk, no intermediate format, no second copy of the data.

A RE-RUN IS SAFE, because a key is idempotent: writing the same object again yields the same object. That makes a Job with a backoff the right shape and makes “did it finish?” answerable by listing both sides rather than by trusting an exit code — the destination is a bucket, so the census is a LIST and the comparison is name-and-size. What a re-run will NOT do is repair a partially written object in place: Mountpoint writes each file once, sequentially, as a whole-object PUT, so an interrupted file is re-written from the beginning rather than resumed.

WHEN THIS IS THE WRONG TOOL. A passthrough mount is not POSIX — no rename, no append, no in-place modification — so it suits a copy and suits nothing that wants to keep editing afterwards. If the workload's own writes must continue against the destination with ordinary file semantics, the destination wants flint-lean (a local tree with a durable path behind it) or flint-lite (an NFS server whose working set is a PVC). Migrating INTO one of those is a different picture: the destination mount is theirs, and the copy step is the same plain code.

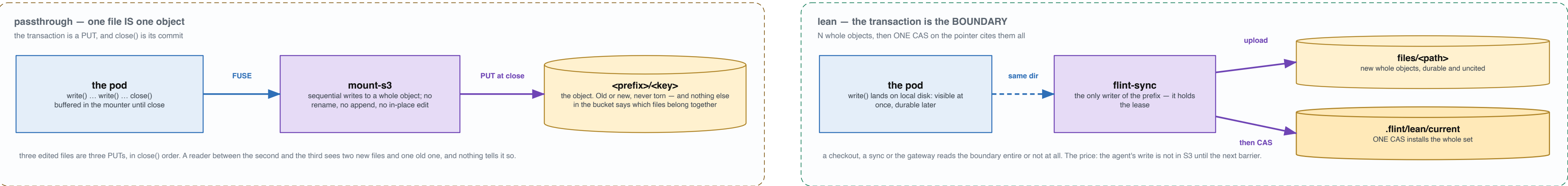
→ data plane — the bytes the copy moves → the object path — the same bytes, as requests - - - - - → control plane — how the mount arrives

What the abbreviations mean

CSI ephemeral inline a volume declared in the pod spec itself — no PVC, no PV, nothing to bind beforehand	volumeAttributes the map on that inline volume. It is where chert.us/mount goes — NOT metadata.labels	NodePublishVolume the CSI call kubelet makes when it schedules the pod. This is when the mount happens	FlintPassthroughMount the CR that names the bucket, prefix, uid and consumers. The POLICY object
chert.us/mount the only key that selects a destination. It names a CR in the pod's own namespace, never a bucket	spec.consumers the ServiceAccounts a mount may be used by. ABSENT MEANS DENY, never means everyone	mount(2) the syscall that attaches a filesystem to a path — the one act here that genuinely needs privilege	SCM_RIGHTS the Unix-socket message that passes an open file descriptor from one process to another
FUSE filesystem in userspace: the kernel hands each file operation to a process instead of a disk	mount-s3 Mountpoint for S3, unchanged upstream. It serves the descriptor the node plugin already mounted	kernel NFS client what serves /mnt/nfs. In-tree, nothing to do with flint — the share is mounted as it always was	flint-s3-broker the one Deployment holding a standing credential. It issues short-lived keys to the worker only
EACCES on every write what a uid mismatch looks like. The CR's uid must match the pod's runAsUser	no rename Mountpoint cannot rename. rsync's default temp-file-then-rename fails; cp and rsync --inplace do not	whole-object PUT each file is written once, sequentially. There is no append and no in-place modification	Job, not Deployment a copy that finishes should be a Job — and a re-run is safe, because keys are idempotent

ACID — passthrough vs lean

Both doors put whole objects in a bucket. Passthrough keeps ONE ledger and inherits S3's guarantees exactly; lean keeps TWO — the objects and a manifest that cites them — and pays for every property it adds with a recovery point on the agent's writes.



The protocol — one shared log, many working copies

Lean is a strongly consistent shared log with eventually consistent working copies: git with automatic push and pull. One conditional write on the bucket orders every boundary; each writer's TICK keeps its tree close to that log, and a UI reads and writes the bucket directly.

